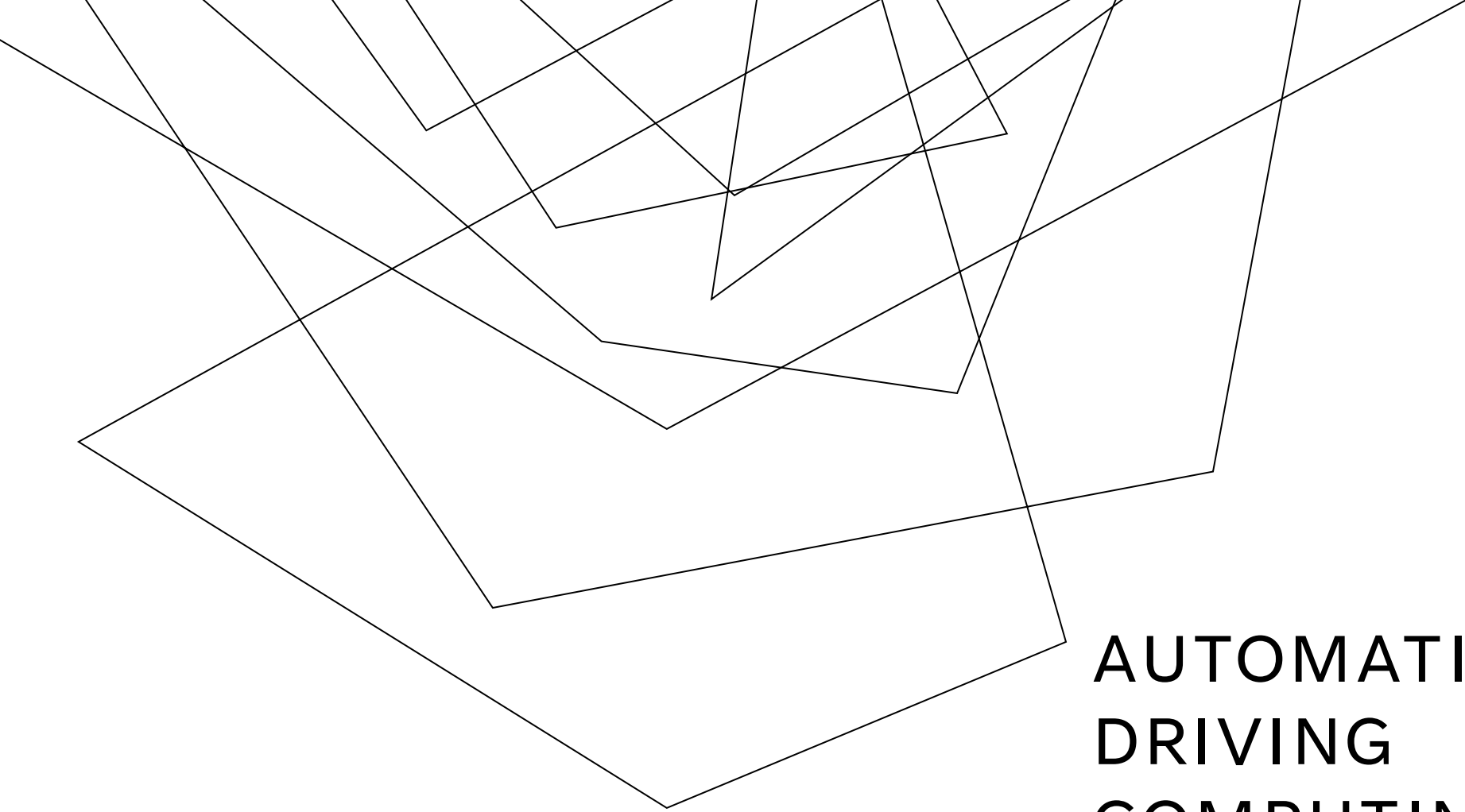


A series of thin, black, overlapping lines forming various geometric shapes like triangles and polygons, creating a complex, abstract pattern in the upper left portion of the slide.

AUTOMATIC DRIVING COMPUTING

7. PERCEPTION (5)





지난 시간 복습

OBJECT DETECTION (3)

1. **Faster R-CNN**
2. **Feature pyramid network**





목차

목차

1. Single-stage object detector



An abstract graphic design featuring two thin, dark gray lines that intersect on a light gray background. One line runs diagonally from the top-left towards the bottom-right, while the other runs from the top-right towards the bottom-left. The intersection point is located to the left of the text.

SINGLE-STAGE OBJECT DETECTOR

SINGLE-STAGE OBJECT DETECTOR

Faster R-CNN: Learnable Region Proposals

Question: Do we really need the second stage?

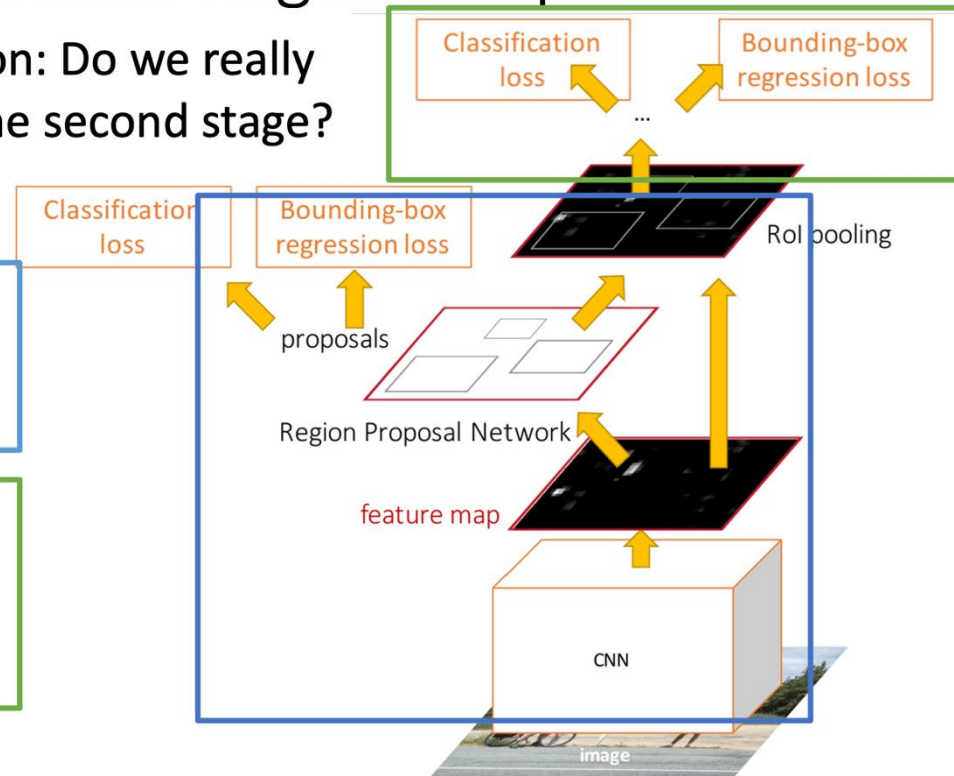
Faster R-CNN is a
Two-stage object detector

First stage: Run once per image

- Backbone network
- Region proposal network

Second stage: Run once per region

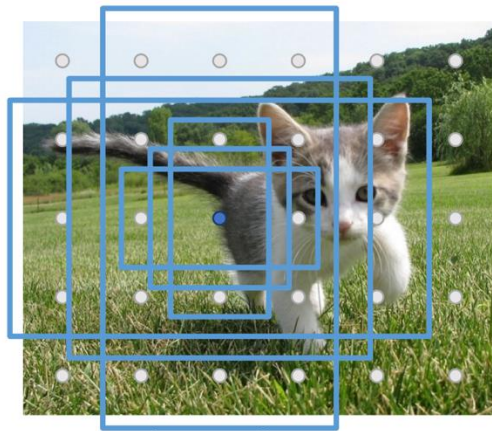
- Crop features: RoI pool / align
- Predict object class
- Prediction bbox offset



SINGLE-STAGE OBJECT DETECTOR

Single-Stage Detectors: RetinaNet

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

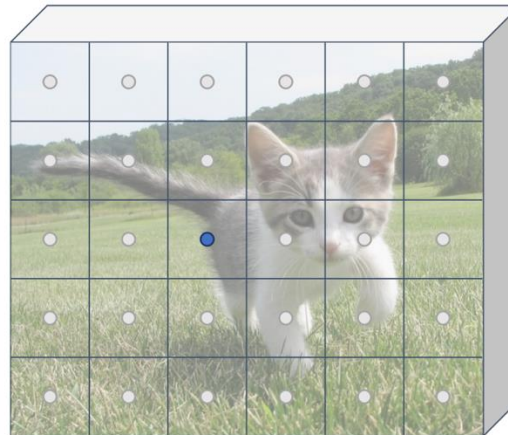


Image features
(e.g. 512 x 5 x 6)



Similar to RPN – but rather than classify anchors as object/no object, directly predict object category (among C categories) or background

Anchor
→ classification
 $2K \times (C+1) \times 5 \times 6$

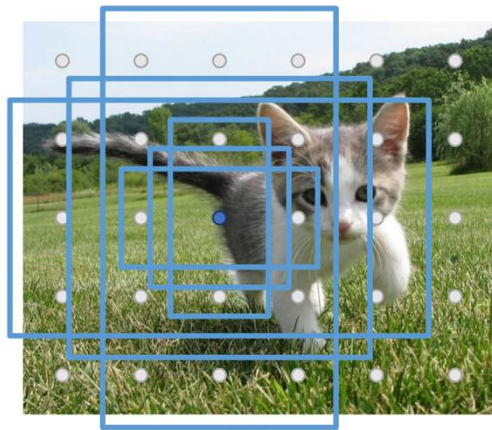
Anchor
→ transforms
 $4K \times 5 \times 6$

SINGLE-STAGE OBJECT DETECTOR

Single-Stage Detectors: RetinaNet

Problem: class imbalance –
many more background
anchors vs non-background

Run backbone CNN to get
features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds
to a point in the input

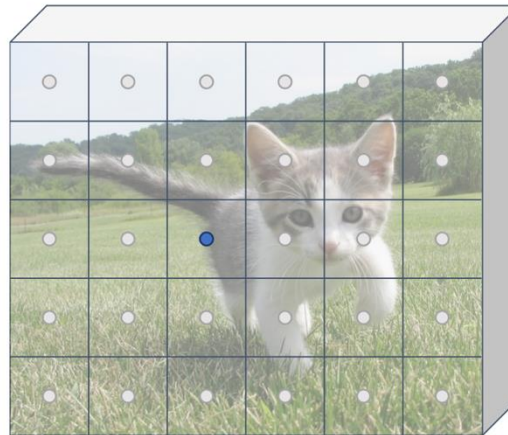


Image features
(e.g. 512 x 5 x 6)



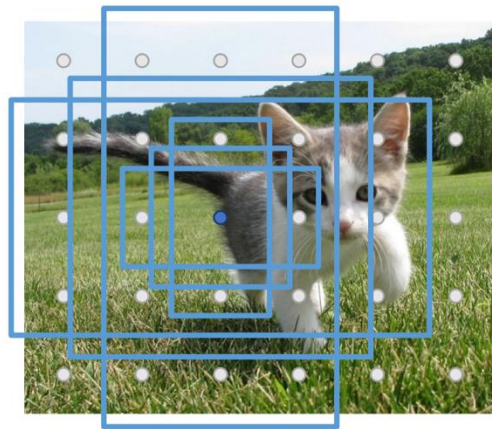
Anchor
→ classification
 $2K \times (C+1) \times 5 \times 6$

Anchor
→ transforms
 $4K \times 5 \times 6$

SINGLE-STAGE OBJECT DETECTOR

Single-Stage Detectors: RetinaNet

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

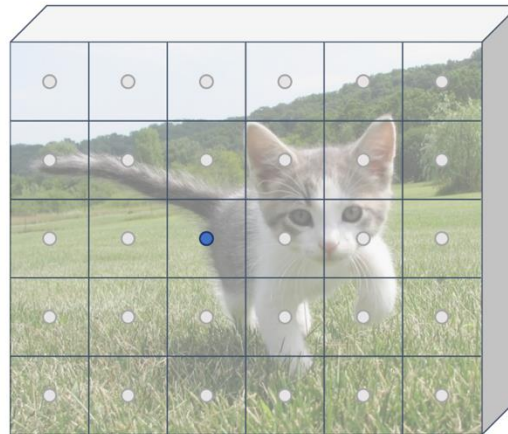


Image features
(e.g. 512 x 5 x 6)



Anchor
→ classification
 $2K*(C+1) \times 5 \times 6$
Anchor
→ transforms
 $4K \times 5 \times 6$

Problem: class imbalance – many more background anchors vs non-background

Solution: new loss function (Focal Loss); see paper

$$CE(p_t) = -\log(p_t)$$

$$FL(p_t) = -(1 - p_t)^\gamma \log(p_t)$$

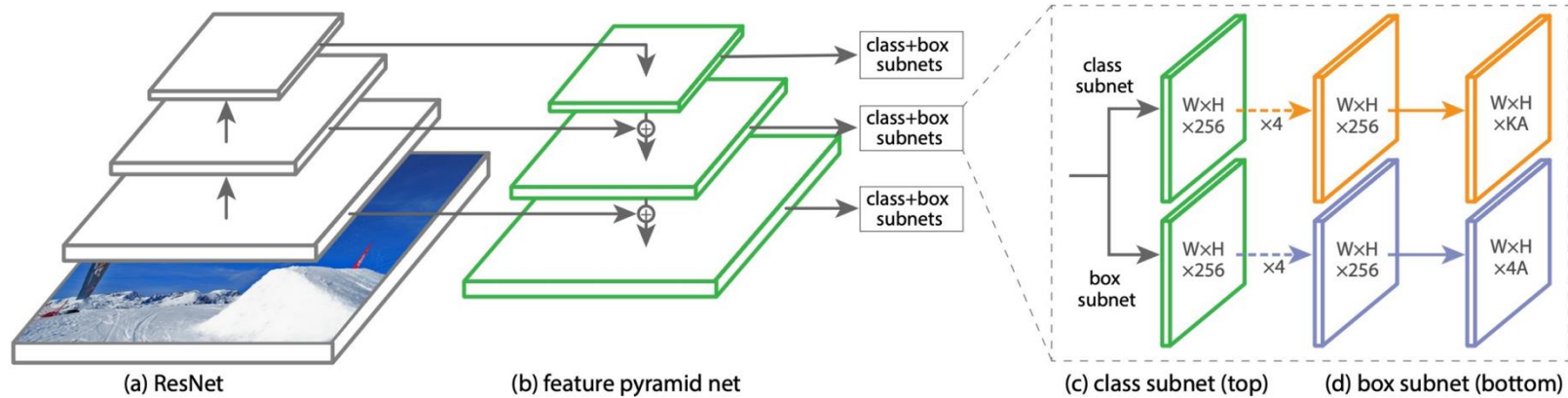
Lin et al, "Focal Loss for Dense Object Detection", ICCV 2017



SINGLE-STAGE OBJECT DETECTOR

Single-Stage Detectors: RetinaNet

In practice, RetinaNet also uses Feature Pyramid Network to handle multiscale



Lin et al, "Focal Loss for Dense Object Detection", ICCV 2017

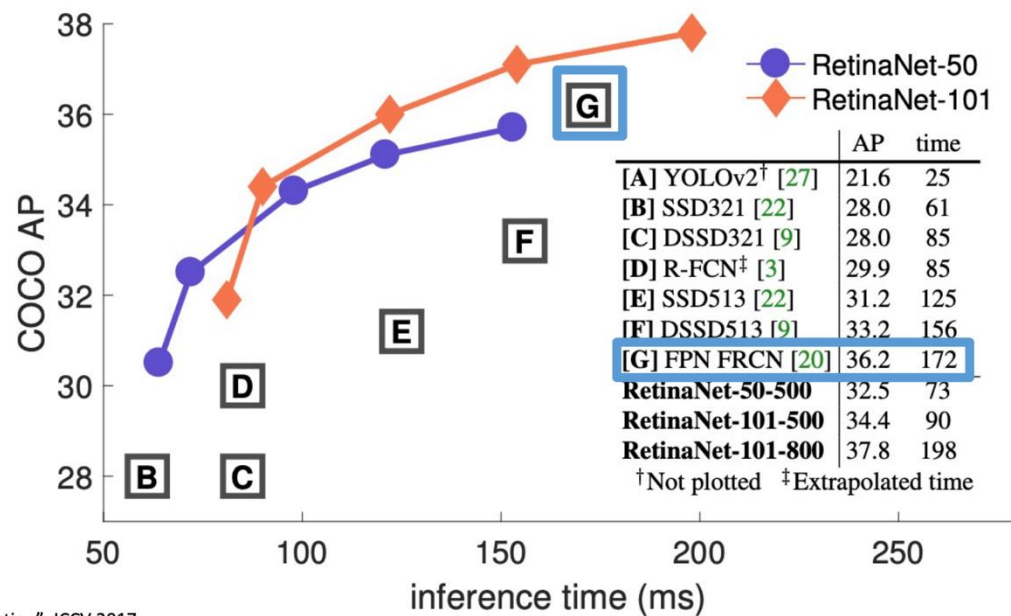
Figure credit: Lin et al, ICCV 2017



SINGLE-STAGE OBJECT DETECTOR

Single-Stage Detectors: RetinaNet

Single-Stage detectors can be much faster than two-stage detectors → 자율주행
시나리오에서 유리



Faster R-CNN
with Feature
Pyramid
Network



SINGLE-STAGE OBJECT DETECTOR

Real-time (45fps) object detector : YOLO family

You Only Look Once: Unified, Real-Time Object Detection

Joseph Redmon*, Santosh Divvala*[†], Ross Girshick[¶], Ali Farhadi*[†]

University of Washington*, Allen Institute for AI[†], Facebook AI Research[¶]

<http://pjreddie.com/yolo/>

Abstract

We present YOLO, a new approach to object detection. Prior work on object detection repurposes classifiers to perform detection. Instead, we frame object detection as a regression problem to spatially separated bounding boxes and associated class probabilities. A single neural network predicts bounding boxes and class probabilities directly from full images in one evaluation. Since the whole detection pipeline is a single network, it can be optimized end-to-end directly on detection performance.

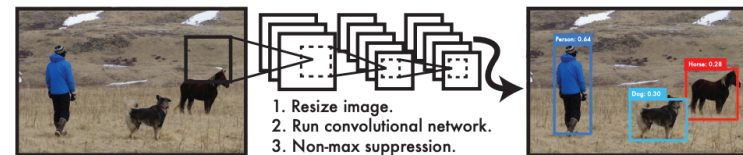


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

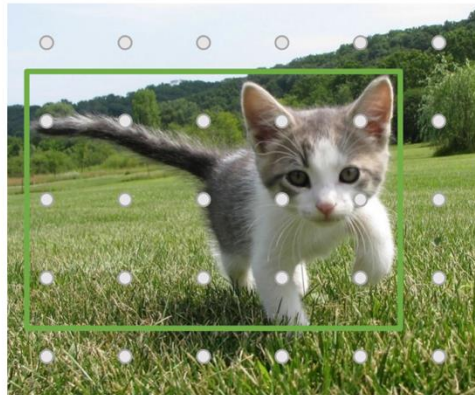


SINGLE-STAGE OBJECT DETECTOR

Single-Stage Detectors: FCOS

“Anchor-free” detector

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

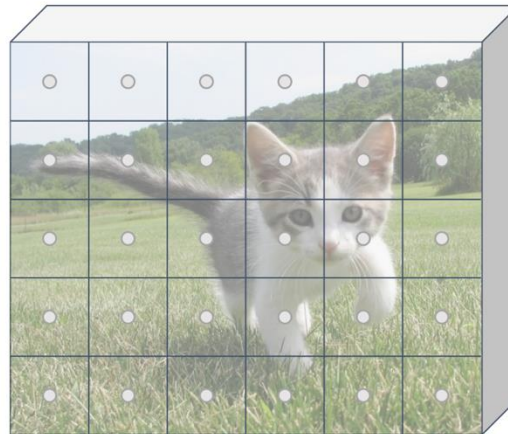


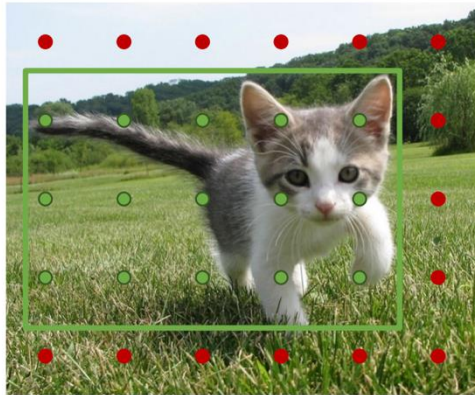
Image features
(e.g. 512 x 5 x 6)



SINGLE-STAGE OBJECT DETECTOR

Single-Stage Detectors: FCOS

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

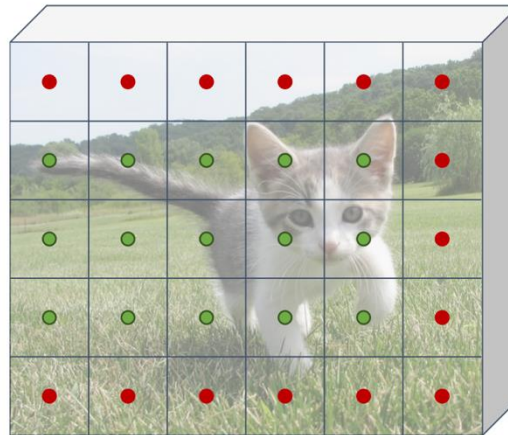


Image features
(e.g. 512 x 5 x 6)

"Anchor-free" detector

Classify points as positive if they fall into a GT box, or negative if they don't

Train independent per-category logistic regressors

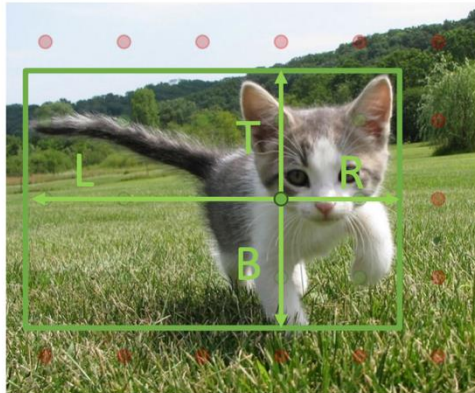


→ Class scores
 $C \times 5 \times 6$

SINGLE-STAGE OBJECT DETECTOR

Single-Stage Detectors: FCOS

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

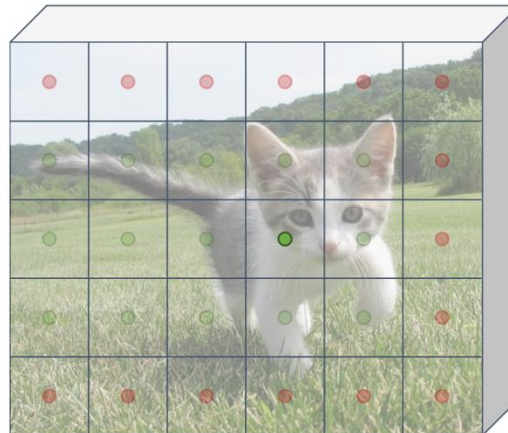


Image features
(e.g. 512 x 5 x 6)

“Anchor-free” detector

For positive points, also regress distance to left, right, top, and bottom of ground-truth box (with L2 loss)



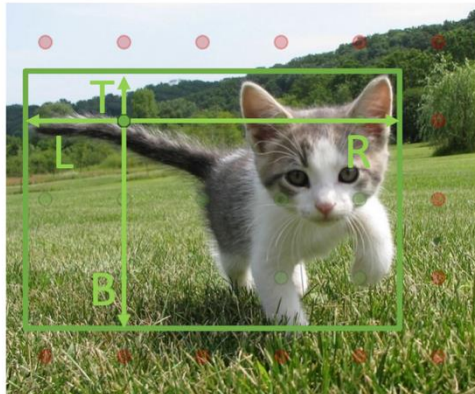
→ Class scores
 $C \times 5 \times 6$

→ Box edges
 $4 \times 5 \times 6$

SINGLE-STAGE OBJECT DETECTOR

Single-Stage Detectors: FCOS

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

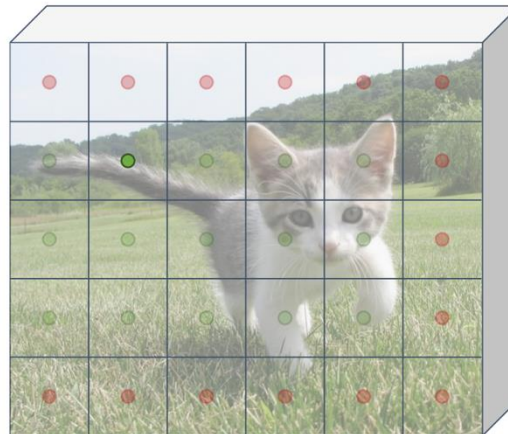


Image features
(e.g. 512 x 5 x 6)

“Anchor-free” detector

For positive points, also regress distance to left, right, top, and bottom of ground-truth box (with L2 loss)



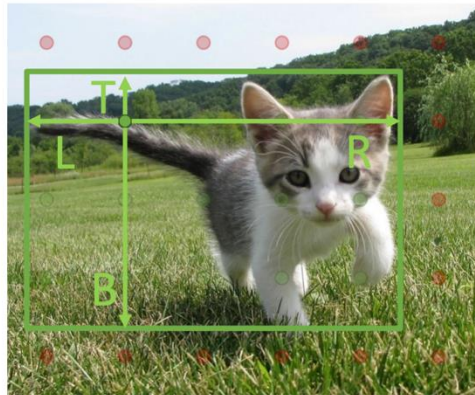
→ Class scores
 $C \times 5 \times 6$

→ Box edges
 $4 \times 5 \times 6$

SINGLE-STAGE OBJECT DETECTOR

Single-Stage Detectors: FCOS

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

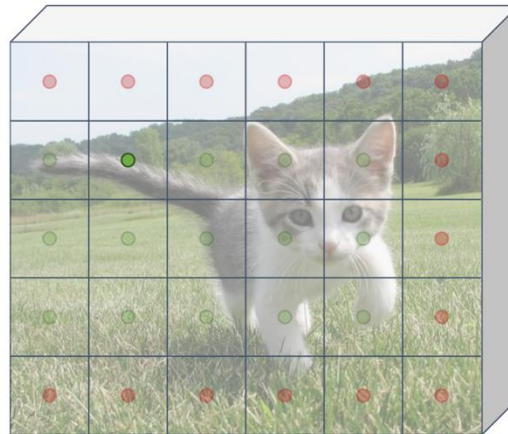


Image features
(e.g. 512 x 5 x 6)



“Anchor-free” detector

Finally, predict “centerness” for all positive points (using logistic regression loss)

- Class scores
C x 5 x 6
- Box edges
4 x 5 x 6
- Centerness
1 x 5 x 6

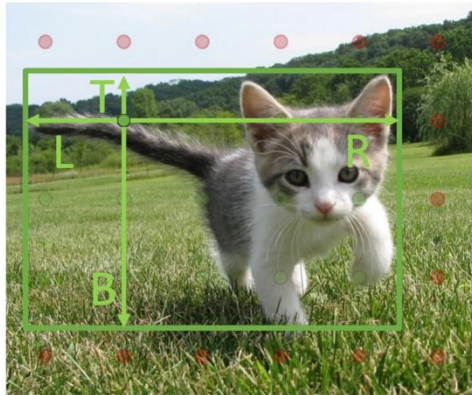
$$\text{centerness} = \sqrt{\frac{\min(L, R)}{\max(L, R)} \cdot \frac{\min(T, B)}{\max(T, B)}}$$

Ranges from 1 at box center to 0 at box edge

SINGLE-STAGE OBJECT DETECTOR

Single-Stage Detectors: FCOS

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)



Each feature corresponds to a point in the input

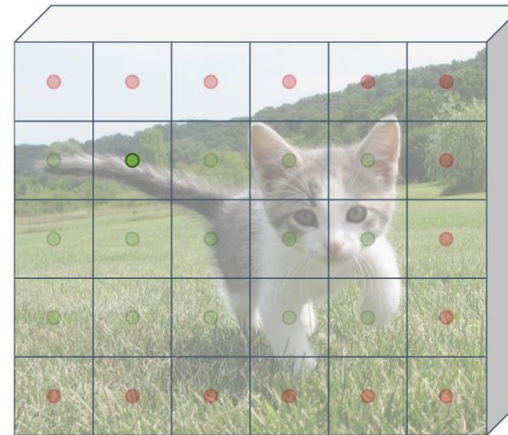


Image features
(e.g. 512 x 5 x 6)

“Anchor-free” detector

Test-time: predicted
“confidence” for the box from
each point is product of its
class score and centerness



- Class scores
C x 5 x 6
- Box edges
4 x 5 x 6
- Centerness
1 x 5 x 6

$$centerness = \sqrt{\frac{\min(L, R)}{\max(L, R)} \cdot \frac{\min(T, B)}{\max(T, B)}}$$

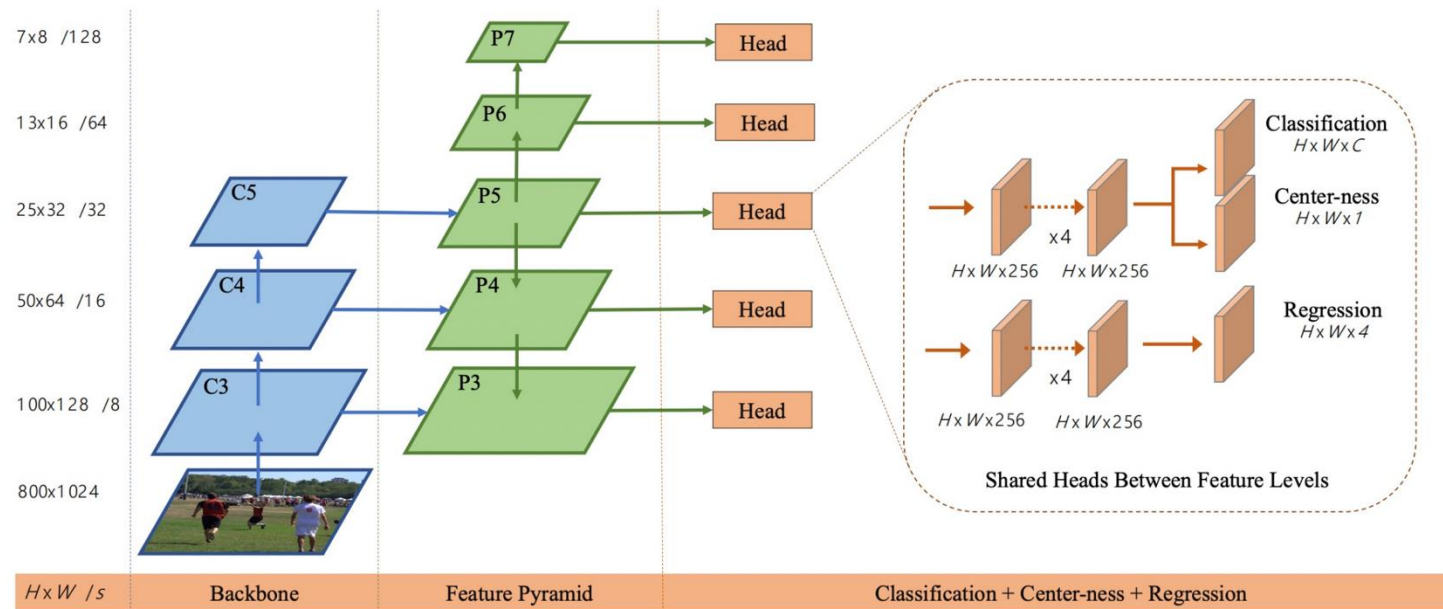
Ranges from 1 at box center to 0 at box edge

SINGLE-STAGE OBJECT DETECTOR

Single-Stage Detectors: FCOS

“Anchor-free” detector

FCOS also uses a Feature Pyramid Network with heads shared across stages



Tian et al, “FCOS: Fully Convolutional One-Stage Object Detection”, ICCV 2019

An abstract graphic featuring two thin, dark grey lines that intersect on a light grey background. One line is oriented diagonally from the top-left towards the bottom-right, while the other is more horizontal, sloping slightly downwards from left to right. The intersection point is located in the upper-left quadrant of the image. To the right of this intersection, the word "SUMMARY" is written in a bold, black, sans-serif typeface.

SUMMARY

SUMMARY

1. Single-stage object detector



NEXT LECTURE

Perception (6) – 2D Semantic / instance segmentation



Two thin, dark gray lines intersect diagonally on a light gray background. One line runs from the top-left towards the bottom-right, and the other runs from the top-right towards the bottom-left. They cross each other in the upper-left quadrant of the image.

QUESTIONS?